DATA STRUCTURE LAB

Array ArrayList

//

length

size ≠ capacity

≠

API

```java
int[] scores = {85, 92, 78};
scores[1] = 95;
int n = scores.length;
```

```java
List<Integer> scores = new ArrayList<>();
scores.add(85);
scores.set(0, 95);
int n = scores.size();
```

`[]` ·

API · □□□□□

size capacity

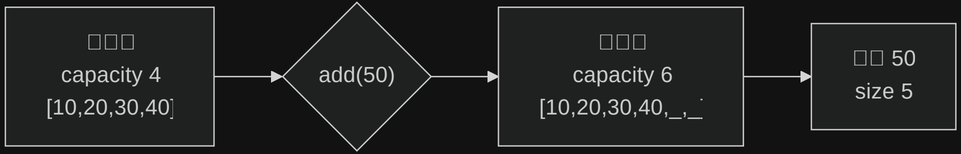
10 | 20 | 30

`size = 3`

`capacity = 4`

capacity List API

□□□□□□□□□□□□□



4 → 6

4 + floor(4 / 2)

□□ 4 □

□□ add □ O(n)

□□ O(1)

□□□□□□□

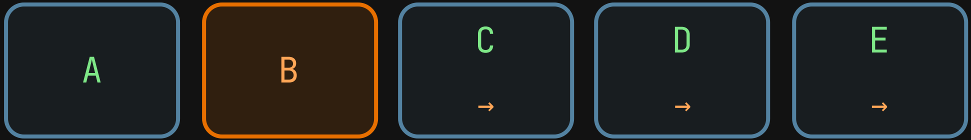
OpenJDK □□□□□ 1.5×□□□□□□□ List □□□□□

□□□□□□□□□□□□

```java
List<Order> orders = new ArrayList<>(incoming);
```

□□□□□□□□□□□□□□□□□□□□□□□□

[illegible]



[illegible] add

[illegible]

index 1 [illegible]

[illegible]O(n)

[illegible]

Array[illegible] length | ArrayList[illegible]size / capacity

0	1	2	3
10	20	30	[illegible]

size = 3 · capacity = 4 · [illegible] 0 · [illegible] 0

40 add 1 insert [illegible]

[illegible]

[illegible] capacity [illegible] add[illegible] insert[illegible]remove[illegible]

primitive □ wrapper

```java
int[] raw = {1, 2, 3};
// □□□ int □
```

Array □□□□ primitive□ `int` □ `double` □ `char`

```java
List<Integer> boxed = new ArrayList<>();
boxed.add(1);           // int → Integer
int n = boxed.get(0);   // Integer → int
```

□□□□□□ `List<int>` □□□ wrapper □ autoboxing□

□□□□□□□

```java
List<Integer> values = new ArrayList<>();
values.add(null);

Integer boxed = values.get(0); // □□□boxed == null
int raw = values.get(0);       // NullPointerException

Integer maybe = values.get(0);
if (maybe != null) {
    int safe = maybe;
    System.out.println(safe);
}
```

□□□□ get() □□□□□□□ `Integer → int` □□□□□□□

Arrays.asList □□□□□□□

```
List<String> readOnly = List.of("Ada", "Linus");
// set / add □□□□
```

`Arrays.asList` □□□□□□□□ view□□□□□ ArrayList□

[illegible]

```java
import java.util.*;

class Roster {
  public static void main(String[] args) {
    List<String> names =
        new ArrayList<>(List.of("Ada", "Linus"));
    names.add("Grace");
    System.out.println(names);
  }
}
```

[illegible]

1. [illegible] String[]
2. [illegible]
3. [illegible] Monaco [illegible] Java

	Array	ArrayList
`get(index)`	O(1)	O(1)
`set(index)`	O(1)	O(1)
`add`		O(1)
/	O(n)	O(n)
		O(n)

Big-O boxing

□□□□□□□□□□□

□□□□□□

□□ 7 □□primitive □□

`double[]`

□□□□

□□□□□□□

`ArrayList`

□□ 8 × 8

□□□□□□□□□□

`char[][]`

□□□□

□□□□□□

`ArrayList`

□□□□□ primitive □□□□□ Array□□□□□□□□ ArrayList□

[illegible]

size [illegible]capacity [illegible]

[illegible] 1.5×[illegible]

primitive [illegible] API [illegible]

□□

□□□□LinkedList□Set□Map □□□□□

choose by behavior, measure by evidence